Shortening the K215 correctness proofs

Verified local frames, execution footprints, and bounded scheduling

Prepared for Vladimir Reshetnikov and the ProveIt repository

22 October 2026 (Pacific time)

Abstract

The most useful next simplification of the existing K215 proofs is to share semantic runtime contracts and scheduler arguments while keeping the exact machine and endpoint statements fixed. Several earlier proposals are already present in the current source, so repeating their original reductions would misstate the remaining work. We review commit `cbf94cb552a3b49509e0eee80e7455d4eb5974c6` and provide concrete, separately compiled components. The main Lean component proves a guarded append-left frame theorem for the maintained symbolic executor and verifies all 35 local bodies proposed in the earlier round-13 packet. These yield whole-configuration positive execution contracts for arbitrary admissible prefixes, gap parameters, and register files. A fresh finite audit binds their 1,422 intended occurrences to the current source and decoder; a separate Lean theorem proves all 35 source actions preserve every path prefix. A Coq component proves exact execution replay under agreement on the rows actually read, with a checked adapter to the current Rocq runtime. A second Lean component strengthens scheduler reachability by a triangular potential and removes the between-candidates premise from completeness of the current K215 enumerator. The contributions are reusable proof components and an integration plan; they are not a fresh build of the complete ZFC endpoint or a smaller machine.

Contents

1 Recommendation and exact scope **2**

2 The actual starting point **3**

3 The guarded frame theorem **4**

4 Thirty-five verified K215 templates **6**

5 Execution footprints for the Rocq runtime **8**

6 One scheduler proof, with a bounded strengthening **9**

7 How to integrate the components **11**

8 Reproduction and evidence **12**

9 Further research questions **13**

A The local-template inventory **14**

B Artifact map and integration status **15**

1 Recommendation and exact scope

The immediate recommendation is to make the runtime proof consume semantic edge contracts, instantiate the already available decoder prefix once, and discharge repeated control suffixes through the frame theorem proved here. Keep the existing literal executor as a fallback producer of contracts. This changes where a proof is shared, rather than changing the theorem to be proved. In parallel, give all enumerators one scheduler interface and make Rocq runtime reuse depend on explicit transition-row agreement.

The implementation supplied with this article makes progress on three well-defined boundaries:

1. **Lean local frames.** A symbolic step commutes with appending an unread left frame, provided the step does not read an empty explicit left fragment. Fixed local runs lift to complete configurations. Existing Home correctness supplies positive progress. All 35 concrete bodies are checked against the current formal-label K215 table.
2. **Rocq execution locality.** An instrumented executor records the transition rows read during a bounded symbolic execution. Agreement on that footprint preserves the exact result for the same target predicate, fuel, and initial progress Boolean. A compiled specialization identifies the parameterized executor with the existing production executor.
3. **Lean scheduling.** One potential proves reach-or-halt from an arbitrary partially processed candidate. The concrete adapter retains the current direct `build_c222` compiler and proves completeness from every enumerator state, with a derivability invariant required only for soundness.

The first idea and its 35 templates were identified by the repository's round-13 left-frame proposal; that packet explicitly left the generic Lean proof uncompiled. Our contribution is its checked realization against the current executor, plus the precise universal contracts and fresh source binding audit. Table locality is a cross-lane realization of a familiar principle already proposed on the Lean side. The scheduler abstraction is inherited from Source194; the triangular bound and current-source adapter are additional work. These distinctions are essential to the provenance of the result [7, 10, 8].

1.1 The machine and the logical endpoint

The accepted machine has 215 working states, two tape symbols, a separate halt, blank input, and a two-way infinite tape. Initialization is included in the state count. Its dense table is identified by SHA-256

`fdbcef0825886035c3091c8537a69535bc`
`96f9e265f0ae45863543d40ae70090.`

The source site file has 4,751 leaves and maximum path length 47. There are 6,176 branch outcomes, since each of the 1,425 decrement sites has two outcomes. The exact counts come from the supplied read-only snapshot inventory and the repository's literal inputs [1, 2].

The existing endpoints have the form

$$\mathrm{Halt}(M_{215}, \mathrm{blank}) \quad \Longleftrightarrow \quad \text{ZFC has a finite derivation of falsity.}$$

Negating both sides gives the corresponding nonhalting/consistency statement. Consistency is not a premise. The Lean theorem is `KEndpoint215.blank_halts_iff_zfc_refutation`; Rocq exposes `tm215_halts_iff_ZFC_inconsistent` and its negated form [3, 4].

The repository explicitly distinguishes these external proof-assistant theorems from a derivation represented in ZFC's own proof syntax. This report preserves that distinction. None of the new components constructs an internal PA or ZFC proof of the equivalence. Obtaining such an internal endpoint remains a separate formalization task; a compilation or an assumption report alone does not supply it [2].

2 The actual starting point

2.1 Source inspection must precede comparisons

The repository has extensive earlier simplification work. The phase-15 overview records a 176-module Lean endpoint, shared recursion and quotient syntax, a smaller PowerSets import boundary, and a shortened dense bridge. However, the current source contains later changes too. A fresh static import census at the pinned commit finds **174 project modules, 1,752,650 source bytes, and 37,648 lines** in the ordinary import closure of `BusyBeaver.ZFC.KEndpoint215`. This census resolves the libraries through the root Lake configuration and includes the bounded-consistency dependencies. It excludes the compiler's own libraries. It is a source inventory, not a fresh full compilation or a declaration-level dependency proof.

Table 1 lists particularly relevant observations. The byte counts measure complete files, including comments and generated declarations. They must not be added to historical savings attributed to earlier packets.

Current source	Bytes	Consequence for this proposal
`KSource215Program.lean`	6,867	The tree is already computed from the program; do not propose removal of the old 122,874-byte tree literal as new work.
`KSource215Data.lean`	116,343	Normalized correspondence is already represented compactly.
Rocq `Source215.v`	832	The current source theorem is a 14-line facade over the normalized checker.
Rocq `NormalizedCert215.v`	109,125	The 1,426-entry normalized certificate is already present.
`EnumPerm.lean`	23,176	The current compiler already uses `build_c222` directly.
`KRuntimeCore.lean`	22,804	A stable 562-line symbolic-executor interface is available for frame lifting.
`KSim.lean`	16,107	The maintained endpoint still consumes its existing whole-site coverage interface.

Table 1: Selected current-file measurements, reproduced by `snapshot_inventory.py`. These are not claimed savings from this work.

The original full-build records report 204 freshly compiled Lean project modules and 237 Rocq sources, with a recursive Rocq kernel check. Later checkpoints retain incremental dependency bindings. We reviewed those records as repository evidence; we did not replay the complete historical build. Current source presence and an earlier successful receipt are different facts [2, 5].

2.2 What should be reused immediately

The following ideas should be credited and retained: the normalized source checker, one-sided correspondence with symmetry, direct compilation in the algebra of block contracts, the representation-

independent implication core, the shared semantic interfaces, and the existing Home return theorem. Likewise, Source193 already provides an all-leaf semantic residual component; it is not limited to the earlier two-outcome example. Its reported component build does not establish that the current named ZFC endpoint has adopted it [5, 7, 9].

One integration hazard is particularly concrete. The older Source194 enumerator snapshot uses the intermediate `CDom` compilation route; the current `EnumPerm` uses `build_c222`. Copying the old file wholesale would undo a later simplification. Scheduler changes should be transplanted as small interfaces or adapters, retaining the current compiler. The supplied adapter does exactly that.

3 The guarded frame theorem

3.1 Why a frame lemma fits the existing representation

The maintained Lean runtime represents a symbolic configuration by

$$s = (q, o, L, R).$$

Here q is the state, o records whether the head is on the left or right list, and L, R are lists of symbolic segments. A segment is a literal bit, a run of length $an + c$, or an encoded register file. The interpretation `conc n vs s` turns this into a complete zipper configuration for a natural gap parameter n and register list vs.

The executor `sstep M br s` returns either failure or a pair (t, m), where m records definite positive progress. One symbolic operation may be a concrete transition, a whole run scan, an orientation change, or a register-file excursion justified by `BoxLemmas M`. Thus symbolic-step counts are not ordinary Turing-machine time bounds [6].

Definition 3.1 (Append-left frame)**.** For a list F of symbolic segments, set

$$A_F(q, o, L, R) = (q, o, L +\!\!+ F, R).$$

Call s *left-safe* if it is not of the form $(q, \text{left}, [], R)$.

The frame is appended at the far end of the explicit left fragment. Its contents are unrestricted for the local step lemma. The safety condition ensures that the next step cannot read the appended frame. It may expose the frame at its output, where a subsequent local step must pass the guard again.

Theorem 3.2 (One-step frame commutation)**.** *For every table M, branch condition br, left-safe symbolic configuration s, and frame F,*

$$\text{sstep}_{M,br}(A_F s) = \text{map}\big((t, m) \mapsto (A_F t, m)\big)(\text{sstep}_{M,br}(s)).$$

In particular, a successful step preserves its exact successor and progress bit after appending the frame.

Proof. Inspect the head orientation and the first segment. For a nonempty left list, a bit reorientation touches its head, and a run scan or peel touches only that run and the adjacent right list. Appending F commutes with each list operation. A left-side register-box case fails on both sides.

When the head is on the right, a literal transition, a run scan or peel, and an oracle excursion either leave the left list unchanged or prepend segments to it. These operations commute with appending F. The same transition table and branch oracle are consulted in both executions, so all case conditions agree. The only case in which appending a frame changes which segment is read is an empty left list with the head on the left; the hypothesis excludes it. The progress Boolean is not altered by any of these rewrites. □

The formal theorem is `KLeftFrame.sstep_frame`. Its proof works by case analysis on the existing definition, without replacing the executor, introducing a new segment constructor, or asserting a window-only semantics. The specialization `sstep_addLeft` exposes the successful-step form.

3.2 The guard is necessary

For an empty explicit left fragment, the unframed executor reorients onto a blank cell. If a nonblank frame has been appended, the framed executor instead reorients onto the first cell of that frame. Those symbolic configurations need not agree. Therefore an unconditional frame equation is false. It is also unsound to normalize away trailing zeros from an unframed left fragment and subsequently append a nonblank frame: the discarded zeros would then separate the local fragment from that frame. The supplied proof does neither.

3.3 A finite local run gives a universal semantic contract

Define `runLocal` by a fixed number of symbolic steps. At each positive fuel value it checks left-safety, performs one `sstep`, and recurses. At zero steps it returns the current configuration. It has no early target test and no implicit permission to read through the frame.

Theorem 3.3 (Local replay soundness)**.** *Assume the existing register contracts `BoxLemmas M` and validity of vs for br. If*

$$\mathrm{runLocal}_{M,br}(k, s) = \mathrm{some}(t),$$

then for every $n \in \mathbb{N}$ and every symbolic frame F,

$$\mathrm{Reach}_M(\mathrm{conc}(n, vs, A_F s), \mathrm{conc}(n, vs, A_F t)).$$

Proof. Induct on k. The zero case is reflexivity. In the successor case the successful local computation provides safety, a successful step, and the remaining local run. Theorem 3.2 lifts the step through the frame. The existing `sstep_sound` supplies concrete reachability; compose it with the induction hypothesis. No local positivity assertion is needed. □

This weaker conclusion is deliberate. The final Home operation is already known to make positive progress under the existing `KHome.Rows M` and `KHome.Boundary M` contracts. With those contracts, if Home maps the framed local exit to u, its theorem and Theorem 3.3 give

$$\mathrm{Reach}^{+}_M(\mathrm{conc}(n, vs, A_F s), \mathrm{conc}(n, vs, u)).$$

The implementation is `local_home_sound`. Keeping positivity at this single composition boundary avoids proving a redundant moved-bit invariant for every local body.

4 Thirty-five verified K215 templates

In this section M_{215} denotes the maintained sparse formal-label table `KTableCore.tm215`. The existing dense bridge relates it to the literal 215-state machine. The new component uses the sparse runtime representation; it does not reprove that dense bridge.

4.1 The exact universal statement

Every listed template has an entry configuration s_t, a fixed local-step count k_t, and a right-list result R_t. The checked equation is

$$\mathrm{runLocal}_{M_{215},\mathrm{none}}(k_t, s_t) = \mathrm{some}(2, \mathrm{left}, [], R_t).$$

Let P be a finite Boolean prefix and let

$$F_P = \mathrm{reverse}(P) \text{ mapped to bit segments } ++ [\mathrm{bit}(1)].$$

The existing Home scan predicate must accept the reversed prefix:

$$\mathrm{scanBound}(0, \mathrm{bits}(\mathrm{reverse}(P))) = \mathrm{some}(j) \quad \text{for some } j.$$

This is the exact formal premise, rather than an informal assumption that the prefix "looks safe." Prefixes ending in zero are permitted when the predicate accepts them.

Theorem 4.1 (Concrete K215 template contract)**.** *For each of the 35 listed templates, every accepted prefix P, every $n \in \mathbb{N}$, and every valid register list vs,*

$$\mathrm{Reach}^{+}_{M_{215}}(\,\mathrm{conc}(n, vs, A_{F_P} s_t),\ \mathrm{conc}(n, vs, (84, \mathrm{right}, [\mathrm{bit}(1)], \mathrm{bits}(P) ++ R_t))).$$

The formal theorem is `KLeftFrame215.template_prefix`. It checks `KTape.boxLemmas` for the current table and discharges the Home readiness conditions by kernel computation. There are no unproved machine-specific register or Home hypotheses left in this concrete theorem. Its register validity premise includes the existing minimum register-list length. The interpreted exterior is the ordinary blank exterior of the maintained configuration representation; the theorem does not generalize Home to an arbitrary nonblank far-left tape.

The local gap has length $n + 16$. At a source path pc, the existing gap $n + (64 - |pc|)$ is obtained by using the template parameter $n' = n + 48 - |pc|$. The current path-length bound makes this a natural number. The gap change at exit is accounted for by the literal zero segments in R_t, before any source-interface normalization.

4.2 What was checked

The template family contains 29 Continue templates and six Break templates. Their bodies contain 444 symbolic operations in total, with maximum body length 39. Fresh Lean 4.32.0 component builds check 14 distinct modules: three new modules and 11 exact repository dependencies. The Boolean theorem `all_checked` is obtained by kernel reduction of all 35 equations. The universal contract then follows from the generic frame and Home proofs. This is stronger than testing selected concrete prefixes or register values.

The earlier native traces use labels that differ from the maintained formal table. In particular native labels 301, 302, and 254 correspond to formal labels 102, 120, and 230. Native 302 occurs inside these local traces, so translating entry labels alone would be insufficient. The template data used here is the already translated formal-label family, rebound to the current table by the new checks [7].

Theorem 4.2 (Source actions preserve the prefix)**.** *For every one of the 35 listed source-action records, with suffix S_t and result E_t, and every Boolean word P,*

$$\mathrm{action}_t(P +\!\!+ S_t) = P +\!\!+ E_t.$$

The theorem `KLeftFramePaths215.prefix_action` uses the actual `RegisterPath.continuePath` and `breakPath` definitions. Each suffix contains the required delimiters, and each Break increment finishes inside the suffix. Case reduction over the finite list therefore leaves P arbitrary. This theorem has only `propext` in its assumption report. Formally relating the action record to its runtime-template record, and relating their symbolic right lists to the leaf contract, are still integration obligations.

4.3 Coverage accounting

A separate supplied Python audit checks every literal source path against the current 202-node source DAG and 234-role decoder graph. It then checks the intended template assignments, exact routing state, operation and argument, prefix/suffix split, Home prefix acceptance, framed source action, and gap identity. It finds the same 1,422 template uses: 1,326 Continue outcomes and 96 Break outcomes. Of their prefixes, 721 end in zero. Three control outcomes remain literal.

Outcome family	Total	Template uses	Literal fallback
Increment	1,900	0	1,900
Decrement, both outcomes	2,850	0	2,850
Continue	1,327	1,326	1
Break	98	96	2
Halt	1	0	1
Total	6,176	1,422	4,754

Table 2: Exact intended source coverage. The 1,422 uses share 35 local bodies.

If these uses are incorporated into semantic coverage, the proposed number of separately expensive residual-body checks becomes

$$35 + 4,754 = 4,789,$$

which is 1,387 fewer than 6,176, or approximately 22.46%. All 6,176 coverage and interface obligations remain: many become inexpensive instantiations and exact data checks. The count is inherited from the earlier proposal and freshly audited here; it is not a measured full-build speedup or a net source-size saving.

The distinction between the evidence types is precise. The 35 universal template contracts and their arbitrary-prefix source actions are kernel checked. The assignment of all 1,422 current source occurrences is additionally audited by an executable finite check. The latter must enter the formal source-coverage proof before the template route can replace the existing endpoint's coverage premise. This report does not silently promote a Python equality check to a theorem.

5 Execution footprints for the Rocq runtime

The Rocq source contains candidate-specific copies of runtime modules whose proofs are largely shared. Comparing eight generated candidate directories shows that several important modules become text-identical after the header and candidate namespace are removed. This identifies a maintenance opportunity; it does not prove a reduction in the active K215 import closure.

The smallest useful abstraction is a parameterized symbolic step with an explicit row-read set. It allows a computed certificate or proved local run to be reused when another machine agrees on the rows that execution actually needs, without requiring global equality of the transition tables.

5.1 The generic replay theorem

Let a table be a function $a : K \to A$, and suppose a symbolic step

$$\mathrm{step}(a, s) \in \mathrm{Option}(S \times \mathrm{Bool})$$

has a read list $\mathrm{reads}(s)$. Assume the local property

$$a|_{\mathrm{reads}(s)} = b|_{\mathrm{reads}(s)} \Longrightarrow \mathrm{step}(a, s) = \mathrm{step}(b, s).$$

Use the production executor's convention: zero fuel fails; at positive fuel a target is accepted only when the accumulated progress bit is true; otherwise take a symbolic step and recurse with the disjunction of progress bits.

Define the footprint recursively from execution under a. It is empty at zero fuel or an already accepted target. Otherwise it starts with $\mathrm{reads}(s)$ and appends the remaining execution footprint if the step succeeds.

Theorem 5.1 (Exact replay on a footprint)**.** *If a and b agree on the footprint of execution under a, then the two bounded executions have the same result and the same footprint. The fuel, target predicate, initial configuration, and initial progress bit are identical.*

Proof. Induct on fuel. Zero fuel and target acceptance are immediate. In the remaining case, agreement on the first portion of the footprint and local step locality give identical symbolic successors and progress bits. If the step fails, both executions fail. Otherwise, agreement on the remaining portion permits the induction hypothesis at the common successor. Appending the common first read list also gives equality of the footprints. □

The core theorem is `FootprintReplay.footprint_replay`. It preserves failure as well as success. The footprint is obtained solely from the first table's execution; a second search is not assumed. Duplicated row keys in the list are harmless and can later be deduplicated by a separate finite membership argument.

5.2 A checked adapter to the existing executor

`Runtime215Locality.v` defines `sstepWith` by parameterizing only the four transition-table applications in the current `Runtime230Core.sstep` body. Its `readRows` returns at most one row: the row for a visible bit or run, or the blank row needed by a box-edge fallback. Pure reorientation and successful fixed oracle cases have no table read at this layer.

The specialization equation `sstepWith_original` is definitional. The theorem `execWith_original` identifies the iterated executor with the production one by induction. Applying Theorem 5.1 gives `siteFootprint_replay`. These statements compile against the exact maintained runtime dependency sources.

There is one semantic qualification. An oracle excursion is independent of the table *as a symbolic computation*, but its correctness as a concrete machine run depends on register contracts for the chosen table. Therefore replay equality alone does not transfer the full halting theorem to an arbitrary new machine. The target table must still satisfy the appropriate runtime/register soundness hypotheses. The supplied theorem states exact executor equality and does not conceal those semantic obligations.

The component uses Coq 8.18.0 and eight exact upstream dependency modules plus two new modules. The new theorem assumption reports are closed. This is compatible Rocq-language proof work; it is not a claim that the repository's complete Rocq 9.1.0 endpoint was rerun under its pinned toolchain.

6 One scheduler proof, with a bounded strengthening

6.1 The common structure

The current enumerators repeat `run_inv`, `run_add`, `drain`, and `reach`. Across `EnumPerm`, `EnumDom`, `Enum222`, and `Enum254`, those 16 declarations occupy 209 lines and 9,500 bytes. That is the duplication scope, not a net saving: a shared module and its concrete adapters must be counted too.

Source194 already isolates a useful `Dynamics` interface with the existing step and finite-run functions. Its laws express two cases. Write $p(s)$ for the pending-work rank of the current candidate and $i(s)$ for the next candidate index. If a successful step takes s to t, then

$$p(s) > 0 \Longrightarrow p(t) < p(s) \quad \text{and} \quad i(t) = i(s),$$
$$p(s) = 0 \Longrightarrow i(t) = i(s) + 1.$$

That abstraction supports a qualitative cofinality theorem without a bound on refill size. It should remain the general interface.

The K215 decoder supplies an extra fact:

$$p(s) = 0 \text{ and } s \to t \quad \Longrightarrow \quad p(t) \leq i(s).$$

This permits a simple numerical strengthening.

Definition 6.1 (Triangular scheduler potential)**.** Let $\mathrm{T}(0) = 0$ and $\mathrm{T}(n+1) = \mathrm{T}(n) + n + 1$; equivalently $\mathrm{T}(n) = n(n+1)/2$. For a target N and $i(s) \leq N$, set

$$V_N(s) = \mathrm{T}(N) - \mathrm{T}(i(s)) + p(s).$$

Lemma 6.2 (Strict decrease)**.** *Until $p(s) = 0$ and $i(s) = N$, every successful step stays at index at most N and decreases V_N strictly.*

Proof. If $p(s) > 0$, the index is unchanged and the remaining work decreases. If $p(s) = 0$ and the target boundary has not been reached, then $i(s) < N$. The new index is $i(s) + 1$, and the refill

bound gives

$$\begin{aligned} V_N(t) &\le \mathrm{T}(N) - \mathrm{T}(i(s)+1) + i(s) \\ &= \mathrm{T}(N) - \mathrm{T}(i(s)) - 1 = V_N(s) - 1. \end{aligned}$$

All subtractions are natural-number subtractions with the needed order bounds established first. The Lean proof uses the recursive triangle identity, so no nonlinear arithmetic procedure is required. □

Theorem 6.3 (Bounded reach-or-halt)**.** *For every state s with $i(s) \le N$, there are k and t such that*

$$k \le V_N(s), \qquad \mathrm{run}(k, s) = \mathrm{some}(t),$$

and either the next step at t halts, or $p(t) = 0$ and $i(t) = N$.

Proof. Use well-founded recursion on $V_N(s)$. If the target boundary is already reached, take $k = 0$. If the next step halts, again take $k = 0$. Otherwise perform the successful step, apply strict decrease, and prepend that step to the recursively supplied prefix. The natural strict decrease pays for the extra step in the bound. □

The theorem counts successful enumerator steps. If the result is a state whose next step halts, that last halting transition requires one additional step. It does not bound the binary machine's physical runtime. The result assumes neither consistency nor an invariant on the logical working payload. It also does not show that a target boundary is reached unconditionally: earlier halting is an explicit alternative.

6.2 Specialization to the current K215 enumerator

For `EnumPerm`, $p(s) = s.\mathtt{pl}$ and $i(s) = s.\mathtt{np}$. The existing `step_eq` and `decode_le` prove the local laws and refill bound. The wrapper uses the original `enumStep` and `run`; it does not introduce a competing executable semantics. The concrete theorem is `find_candidate_bounded`.

The same component supplies a useful stronger completeness statement:

Theorem 6.4 (Completeness from any scheduler position)**.** *For every current K215 enumerator state s,*

$$C_{222}(\mathrm{F}) \quad \Longrightarrow \quad \mathrm{HaltsFrom}(s).$$

Consequently, under the existing derivability invariant on s's payload,

$$\mathrm{HaltsFrom}(s) \quad \Longleftrightarrow \quad C_{222}(\mathrm{F}),$$

without the old premise $s.\mathtt{pl} = 0$.

Proof. Compile the derivation with the current `build_c222`, obtaining a block p with the established contract. Prepend the existing reset command and append the existing terminating suffix. The encoded program index N is at least the starting next-candidate index by `encL_ge`.

Apply Theorem 6.3 from the original state, even if a candidate is partly processed. If an earlier halt is reached, completeness is already proved. Otherwise the run reaches the boundary that fetches the selected encoded program. The existing `program_none` works from every payload, and `exec_list` executes the program there. Compose the prefixes with the original `run_add`. For the equivalence, use the existing `sound` theorem in the other direction, retaining its invariant. □

This strengthens the maintained `complete` and `halts_iff` interfaces, both of which currently require a between-candidates state. It is an exact concrete result about the current enumerator. Arbitrary-state completeness was already proposed in Source194 and related earlier work; we claim its checked attachment to the current compiler, not its mathematical discovery. The triangular potential is an optional quantitative refinement of the more general Source194 scheduler.

7 How to integrate the components

7.1 Preserve the final theorem and change its producers

The endpoint theorem itself is already short. Its complexity lies in the objects and semantic results it composes. A useful architecture has the following interfaces:

Boundary	Producer	Consumer
Literal table identity	dense/sparse bridge and exact row facts	concrete runtime semantics
Symbolic step soundness	row contracts and register contracts	generic frame/replay theorems
Source-edge semantics	decoder prefix, local templates, literal fallback	source/TM simulation
Enumerator cofinality	shared scheduler and local laws	direct block compiler completeness
Logical calibration	existing calculus/ZF/ZFC development	unchanged final endpoint

The consumer of a semantic edge contract should not know whether its proof came from a long literal replay, a shared template, or a hand-proved routine. This is the main simplification: the proof structure follows the operation's meaning, while exact tables remain checked data.

7.2 A concrete sequence of patches

Patch 1: add the frame library and the 35-template module. These are additive and already checked independently. Keep their theorem statements independent of the source-coverage generator. The exact `safeLeft` guard and final Home premise belong in the public interface.

Patch 2: connect the semantic coverage interface. Transplant the Source193 residual coverage interface into the current `KSim`/endpoint route, preserving the existing Boolean checker as an adapter. Do not replace an entire older source snapshot: later Home, lookup, and compiler changes must survive. Check the old API clients while this interface is introduced.

Patch 3: turn the occurrence audit into formal coverage. For each source occurrence, check its exact path split, routed state, operation, source-action equation, gap substitution, and Home guard. Use the template theorem when its verified predicates hold and the literal residual checker otherwise. The finite table must still cover every source leaf and every decrement outcome. Only then replace the old coverage producer in the simulation theorem.

Patch 4: share scheduler proofs through adapters. Retain the existing executable definitions. Move generic run composition and cofinality into one module, instantiate local laws for each enumerator, and use the new arbitrary-position completeness theorem when helpful. Keep the current direct compiler. The triangular bound should be a separate theorem, so enumerators without its refill premise can still use the qualitative interface.

Patch 5: parameterize the Rocq runtime at the table boundary. Use the supplied equality and footprint theorems as the first small change. Separate the symbolic executor from the concrete

table module, then add semantic register/routine hypotheses at the existing soundness boundary. Candidate-specific files should eventually contain table data, row checks, boot facts, and a short instantiation. This larger refactor still needs a fresh Rocq 9.1.0 endpoint build.

7.3 The acceptance condition

For each adopted patch, preserve the exact type of both halting and nonhalting endpoint declarations and their machine data. Record successful compilation of affected descendants and endpoint clients. Check the actual assumption reports: the established Lean endpoint profile is `propext`, `Classical.choice`, and `Quot.sound`; the recorded Rocq endpoint reports are closed. Keep generator reproduction separate from proof compilation, and retain failed attempts with their actual status. These are the repository's existing evidence distinctions, not new mathematical assumptions [2, 5].

A faster local check is not automatically a faster full build. Measure kernel-check time, proof-object size, source size, and the source/import closure separately. In particular, an additive pilot can increase source size while enabling a later replacement that reduces repeated proofs. No net endpoint-size reduction is claimed for the present additive packet.

8 Reproduction and evidence

The archive contains the article source and PDF, the three component source packets, portable build scripts, raw accepted compiler logs, dependency manifests, source inventories, and the finite assignment audit. The formal packets vendor only their selected source dependencies, with inherited licenses and notices. Compiled objects are regenerated locally.

Component	Toolchain	Meaning of a successful check
Local frames and templates	Lean 4.32.0	Generic frame/replay/Home theorems and all 35 universal K215 template contracts.
Execution footprints	Coq 8.18.0	Generic replay and its equality/locality adapter to the exact existing Rocq runtime.
Bounded scheduler	Lean 4.32.0	General triangular bound and the concrete current-enumerator theorems.
Occurrence audit	Python 3	Exact finite bindings and intended coverage of 1,422 uses; not a kernel proof of source coverage.
Snapshot inventory	Python 3	Current source identities and ordinary import closure; not a fresh full endpoint build.

Table 3: Evidence scopes. Compiler receipts and raw logs accompany the source.

The supplied build scripts perform serial proof invocations and write their results into chosen output directories. The package README gives the exact commands. Lean's own release libraries and the installed Coq standard library are explicit toolchain boundaries. No downloaded discovery script, external finite test, or unproved declaration is used as a premise of the new formal theorems.

All three concrete scheduler theorems report the ordinary Lean foundational profile `propext`, `Classical.choice`, and `Quot.sound`. The concrete template theorem has the ordinary Lean foundational profile `propext`, `Classical.choice`, `Quot.sound`. The literal `all_checked` theorem reports only `propext`. The new generic Coq replay and actual adapter theorem reports are closed under the global context. Assumption reports should always be read for the exact declarations

named in the logs, rather than generalized to every imported library.

9 Further research questions

1. **Can the 1,422 occurrence bindings be discharged by one small reflected checker?** The next concrete milestone is a theorem turning the existing exact assignment predicate into source-edge semantics, followed by kernel reduction of its finite coverage table. This would complete the formal bridge between the verified templates and the source simulator.
2. **How much reuse is available in increment/decrement bodies?** The remaining 4,750 register-operation outcomes dominate the literal family. Seek a small set of register-relative contracts, with branch premises carried explicitly. A new contract should preserve complete configurations, including register tails, head position, and gap changes.
3. **What is the smallest useful semantic template family?** The 35 bodies need not be minimal. Several implement the same source operation from different routed states. A quotient by certified entry stuttering or by identical residual effects could reduce the family, but the extra adapter proofs must be charged against the saving.
4. **Can a single data format serve both assistants?** Share literal templates, row footprints, source assignments, and versioned hashes. Prove each parser/checker sound in its own assistant. This avoids making either compiler or an unproved cross-assistant translation a new premise.
5. **Which scheduler abstraction is simplest in actual clients?** Compare the Source194 qualitative cofinality theorem with the stronger bounded theorem. The former needs fewer hypotheses. Both share the drain/reach reasoning; the latter additionally gives explicit prefix bounds. The right default should be decided by client source size and clarity, not by theorem strength alone.
6. **Can active theorem dependencies be reduced beyond import dependencies?** The current 174-module census measures whole files. Small semantic interfaces and declaration-level dependency analysis may reveal unused calibration or compatibility layers, but any cut must preserve the final theorem's meaning and the existing API clients.
7. **How should the internal ZFC endpoint be formalized?** Identify a weak arithmetical theory adequate for the runtime and compiler proofs; represent the logical calibration and its proof translation explicitly. This should produce an internal derivation of the intended arithmetical equivalence rather than relying on external model-theoretic reasoning being informally transferable.

A The local-template inventory

The body count below is a count of guarded symbolic operations, not binary TM transitions. The entry state uses the maintained formal labels. The source suffix/result columns summarize the intended source action; the kernel theorem's exact physical exit is its literal `exitRight` list.

ID	Operation	Suffix	Entry	Steps	Result
0	CC3	10111	62	12	10
1	CC2	111	13	8	1
2	CB1	1001	108	7	11
3	CC2	111	42	5	1
4	CC2	111	138	11	1
5	CC2	111	141	5	1
6	CC2	10000011	42	15	100000
7	CC2	1011	57	7	10
8	CC3	100111	62	14	100
9	CC2	111	57	5	1
10	CC3	10111	80	21	10
11	CC2	111	72	8	1
12	CC2	111	126	5	1
13	CC2	10011	42	9	100
14	CC2	111	47	5	1
15	CC3	1000111	83	13	1000
16	CC2	111	139	5	1
17	CC2	1000011	42	13	10000
18	CC3	100000111	83	17	100000
19	CC3	10000111	83	15	10000
20	CC2	1011	126	7	10
21	CC3	100111	83	11	100
22	CC2	100011	42	11	1000
23	CC3	1111	62	10	1
24	CC2	1000101	108	13	1000
25	CB1	0011	103	24	1
26	CC1	101	122	5	10
27	CC2	1101	108	7	1
28	CB1	0110111	103	39	100
29	CC1	11	122	3	1
30	CC2	1011	42	7	10
31	CC2	10000101	108	15	10000
32	CB1	00111	103	29	1
33	CB1	0011111	96	39	1
34	CB1	0011	162	24	1

B Artifact map and integration status

`lean-runtime/`
The generic append-left library, the concrete template family, exact dependency sources, compiler receipts, and finite source-assignment audit. The README identifies any additional path-framing results and their exact scope.

`rocq-locality/`
`FootprintReplay.v`, `Runtime215Locality.v`, eight exact dependency sources, build script, closed assumption logs, and recursive checker evidence.

`lean-scheduler/`
The inherited Source194 dynamics interface, new bounded potential, concrete K215 adapter, 14 exact production dependencies, and reproduction evidence.

`audit/`
The pinned static source inventory and its script, plus quantitative comparisons used in the article. The files state their own evidence scope.

`README.md` and `VERIFICATION.json`
Commands, source identities, final component statuses, known integration obligations, and pointers to raw logs. The complete ZFC endpoint is marked as not rebuilt in this packet.

References

[1] ProveIt contributors. *Accepted K215 candidate, literal tables, source, and attribution.* Pinned repository revision used throughout this report. Candidate README; attribution notice.

[2] ProveIt contributors. *Formal correctness of the exact 215-state machine.* Verification record, with separate Lean and Rocq build records and stated internal-proof limitation.

[3] ProveIt contributors. *KEndpoint215.lean.* Exact Lean endpoint source.

[4] ProveIt contributors. *CorrectDense215.v.* Exact Rocq dense endpoint source.

[5] ProveIt contributors. *Fifteenth K215 proof-simplification checkpoint.* Checkpoint and retained evidence.

[6] ProveIt contributors, with inherited K230 and m254 runtime work. *KRuntimeCore.lean and KHome.lean.* Symbolic executor; Home correctness.

[7] ProveIt research packet. *Round 13 runtime: factor at the first empty-left Home boundary.* Earlier uncompiled frame proposal, template discovery and formal-label binding. Runtime proposal and data.

[8] ProveIt Source194. *CofinalScheduler.lean.* Generic scheduler source; current K215 enumerator.

[9] ProveIt. *Sources 193–194 reconciliation.* Residual coverage and scheduler status.

[10] ProveIt research packet. *Proof fusion and local reuse.* Compiler fusion and Lean table-dependency transport proposal.

[11] ProveIt. *Computed source tree proposal and current source.* Proposal; current program source.